

56. Une petite mélodie à la main

Une table gravée, et un pas toutes les vingt images.

1. Ce qu'il faut comprendre

`const uint8_t AIR[]` est **gravé dans la cartouche** : il ne prend aucun octet de mémoire de travail, et il ne peut pas changer. C'est exactement ce qu'on veut d'une mélodie.

`sizeof(AIR)` vaut 6, connu à la compilation. **Ajouter une note à la table suffit** — la boucle suit, sans qu'on touche à un nombre.

Ici le programme mène la mélodie et doit y penser à chaque tour. Le tutoriel suivant montre l'autre façon.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <note>` → [0.82.2. La fonction note\(\) — jouer une note](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **722 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      61 o ← la console, à l'allumage - la mise en route de la console
main()         128 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
EcrireNombre   33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles   58 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         224 o ← CopierTuiles - 14 lettres sur 43
Notes          120 o ← note() - la table des notes de musique
"NOTE"         4 o ← texte() - un texte gravé
AIR_0          6 o ← note() - une table gravée
```

Lettres gravées : ENOT0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert

Fonctions du langage appelées :

```
note()   x 1   64 o sur place, dans main() + Notes (120 o), AIR_0 (6 o)
texte()  x 1   11 o sur place, dans main() + EcrireTexte (10 o)
nombre() x 1   10 o sur place, dans main() + EcrireNombre (33 o)
image()  x 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : joue en boucle une petite mélodie de 6 notes, une note
// toutes les 20 images, et affiche le numéro de la note jouée.
//
// Les notes sont rangées dans un tableau AIR. « pas » dit laquelle jouer.

#include <texte>    // écrit un texte à l'écran
#include <note>    // joue une note
#include <nombre>  // écrit un nombre en chiffres
```

2. UNE TABLE GRAVÉE DANS LA CARTOUCHE — AIR

```
const uint8_t AIR[] = { D04, MI4, SOL4, D05, SOL4, MI4 }; // 6 notes : AIR[0] à AIR[5].
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

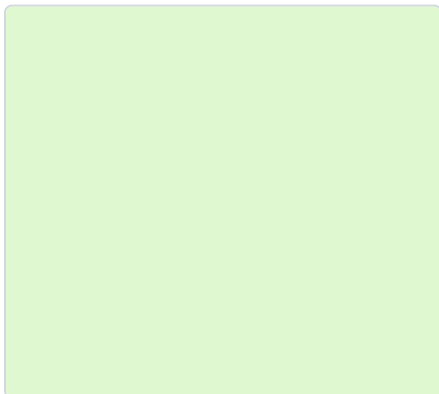
```
int main() {                                // Le jeu commence ici.
    uint8_t pas = 0;                        // La note suivante à jouer (0 à 5).
    uint8_t depuis = 0;                    // Les images depuis la dernière note.

    texte(2, 6, "NOTE");

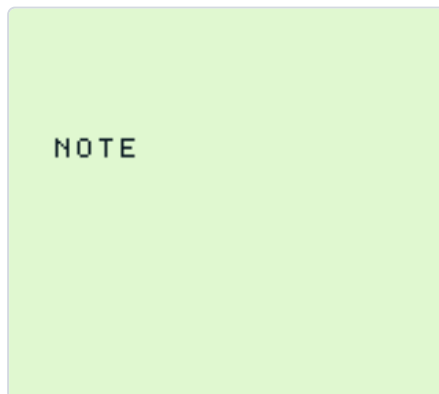
    while (true) {                          // Répète pour toujours :
        depuis++;                          // une image de plus.
        if (depuis >= 20) {                // Toutes les 20 images :
            depuis = 0;
            note(1, AIR[pas], 18, 12);     // joue la note numéro pas (voix 1, 18 images,
                                           // volume 12 ; 18 < 20 : un petit silence entre deux).
            nombre(9, 6, pas, 1);          // affiche son numéro.
            pas++;                          // la suivante...
            if (pas >= sizeof(AIR)) pas = 0; // ... et après la dernière (sizeof = 6), on revient à 0.
        }
        image();                          // Attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

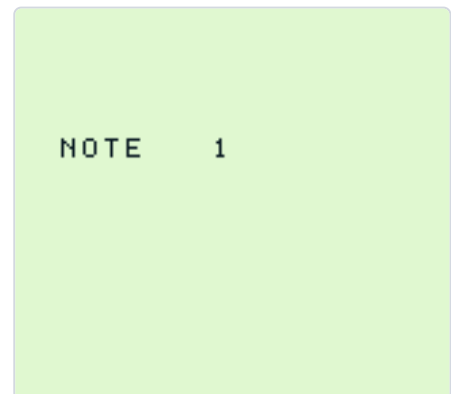
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60

4. Ce qu'on doit voir

Le numéro du pas monte de 0 à 5, puis recommence.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la mélodie avance (2 → 5)

✓ le pas reste dans la table (0)

✓ elle revient au début

✓ et le pas est affiché

Ensuite : **57. Un bruit** — chapitre 8 : Le son. · gameboy3 — écrire une cartouche Game Boy en C++